

CO2-ASSESSMENT TOOL-3

Name: TALARI VISHNUVARDHAN

Reg.no: 192572091

Course name: DESIGN AND ANALYSIS OF ALGORITHMS

Code: CSA0613

Faculty: Dr. F. MARY HARIN FERNANDEZ

Date: 29-07-2026

Question 39

Title

Debugging Maximum Element Search Code (Empty List and Initialization Risk)

Problem Statement

The given Python program is intended to find the maximum element in a list. However, the implementation has a major issue because it assumes that the input list always contains at least one element. If an empty list is provided, the statement `arr[0]` causes an **IndexError**, making the program fail.

The objective is to debug the code by handling empty lists safely, improving robustness through defensive programming, and maintaining correct maximum search functionality.

Original Code

```
1 def find_max(arr):
2     max_val = arr[0]
3     for i in range(1, len(arr)):
4         if arr[i] > max_val:
5             max_val = arr[i]
6     return max_val
```

Problem Analysis

Issue 1: Empty List

- The program directly accesses `arr[0]`.
- If the list is empty (`[]`), Python raises an `IndexError`.

Issue 2: Unsafe Initialization

- The code assumes at least one element exists.
- There is no validation of user input.

Issue 3: Lack of Defensive Programming

- The function should verify whether the list is empty before processing.
- Invalid input should be handled gracefully.

Corrected Algorithm

1. Accept the input list.
2. Check whether the list is empty.
3. If empty, display an appropriate message or return None.
4. Otherwise, initialize the maximum value using the first element.
5. Compare each remaining element with the current maximum.
6. Update the maximum whenever a larger element is found.
7. Return the maximum element.

Python Program (Source Code)

```
main.py
1
2 def find_max(arr):
3     """
4     Returns the maximum element of a list.
5     Returns None if the list is empty.
6     """
7     if not arr:
8         return None
9     max_val = arr[0]
10    for value in arr[1:]:
11        if value > max_val:
12            max_val = value
13    return max_val
14 numbers = input("Enter numbers separated by spaces: ").split()
15 numbers = [int(x) for x in numbers]
16 result = find_max(numbers)
17 if result is None:
18     print("List is empty. No maximum element.")
19 else:
20     print("Maximum Element =", result)
```

OUTPUT

```
Enter numbers separated by spaces: 10 25 7 42 10
Maximum Element = 42
```

Explanation

- The corrected program first checks whether the input list is empty.
- If the list contains no elements, it safely returns None.
- Otherwise, it initializes the first element as the current maximum.
- Every remaining element is compared with the current maximum.
- If a larger value is found, the maximum is updated.
- Finally, the maximum element is displayed.

This approach prevents runtime errors and improves program reliability.

Time Complexity Analysis

Operation	Complexity
Empty list check	$O(1)$
Maximum search	$O(n)$
Overall Time Complexity	$O(n)$
Space Complexity	$O(1)$

Advantages

- Handles empty lists safely.
- Prevents IndexError.
- Uses defensive programming.
- Easy to understand and maintain.
- Efficient with linear time complexity.

Conclusion

The original implementation failed because it assumed that the input list always contained at least one element. The corrected version validates the input before accessing the first element, preventing runtime errors and making the program more robust. The algorithm efficiently finds the maximum element with $O(n)$ time complexity and $O(1)$ space complexity.